

Relatório Técnico – Sistema de Gerenciamento de Pedidos em Java

Arthur de Aquino Anjos, Henrique Estrela Santos, Luan Vinicius Miranda da
Silva, Matheus Kaick Rocha da Fonseca, Rafael Pinto Germano Pereira

Sistemas de Informação - Centro Universitário de Excelência

Bahia, Brasil.

arthuraquino303@gmail.com,henriqueestrela2004@gmail.com,luanvnms2512@gmail.com,
matheuskaick68@gmail.com,rafael.pinto192003@gmail.com

Abstract. *This report presents the refactoring we performed on the FoodDelivery system, focusing on one key change: the application of the Singleton design pattern. The biggest challenge we faced was ensuring that customer, order, and menu item data was always consistent, without the risk of duplication.*

*To solve this, we decided to use the Singleton pattern in our **Repository** class. The idea was to create a single "brain" for the system, a central point that controlled all information to avoid confusion and duplicate data. Here, we'll show how this choice made our code more secure, organized, and prepared for everything we might add in the future.*

Resumo. *Este relatório apresenta a refatoração que fizemos no sistema FoodDelivery, focando em uma mudança chave: a aplicação do padrão de projeto Singleton. O grande desafio que encontramos foi garantir que os dados de clientes, pedidos e itens do cardápio fossem sempre consistentes, sem risco de duplicação.*

*Para resolver isso, decidimos usar o padrão Singleton na nossa classe **Repository**. A ideia era criar um "cérebro" único para o sistema, um ponto central que controlasse todas as informações para evitar bagunça e dados duplicados. Aqui, vamos mostrar como essa escolha deixou nosso código mais seguro, organizado e preparado para tudo que ainda queremos adicionar no futuro.*

1. Fundamentação Teórica

O Singleton é um padrão de projeto bem direto: ele garante que uma classe tenha apenas uma única instância rodando no sistema todo. É a solução perfeita para situações em que você precisa de um ponto de controle único, como um gerenciador de configurações ou, no nosso caso, uma central para guardar todos os dados do projeto.

Segundo Martin (2009), o padrão Singleton deve ser usado com cautela, pois seu uso excessivo pode gerar dependências ocultas e dificultar testes e manutenção do código. Usá-lo pode trazer alguns problemas:

Por ele ser um objeto único que todo mundo compartilha, pode complicar os testes — fica difícil mexer em uma parte do sistema sem que outra interfira, e isso pode esconder bugs. Ele também acaba fazendo “coisa demais”: além de cumprir sua função, precisa se preocupar em ser único, o que quebra a ideia de que uma classe deve ter uma responsabilidade só. E, como é fácil de acessar de qualquer lugar, outras partes do código podem acabar dependendo dele sem perceber, deixando tudo mais amarrado e difícil de mudar ou reaproveitar (MARTIN, 2009).

Para fazer o Singleton funcionar, seguimos duas regrinhas básicas:

No TiaLu Delivery, a gente fez o Repository funcionar como Singleton, e isso ajudou muito. Primeiro, escondemos o construtor da classe, deixando ele `private`, para ninguém conseguir criar outra instância por engano. Depois, criamos uma “porta de entrada” especial: o método `getInstance()`. A primeira vez que ele é chamado, a instância é criada; nas próximas, ele só devolve a mesma, garantindo que todo mundo no sistema use sempre o mesmo objeto.

Isso foi super importante porque assim todas as partes do sistema olham para a mesma lista de informações, aí evita confusão e dados inconsistentes — ou seja, menos dor de cabeça e mais organização.

3. Metodologia

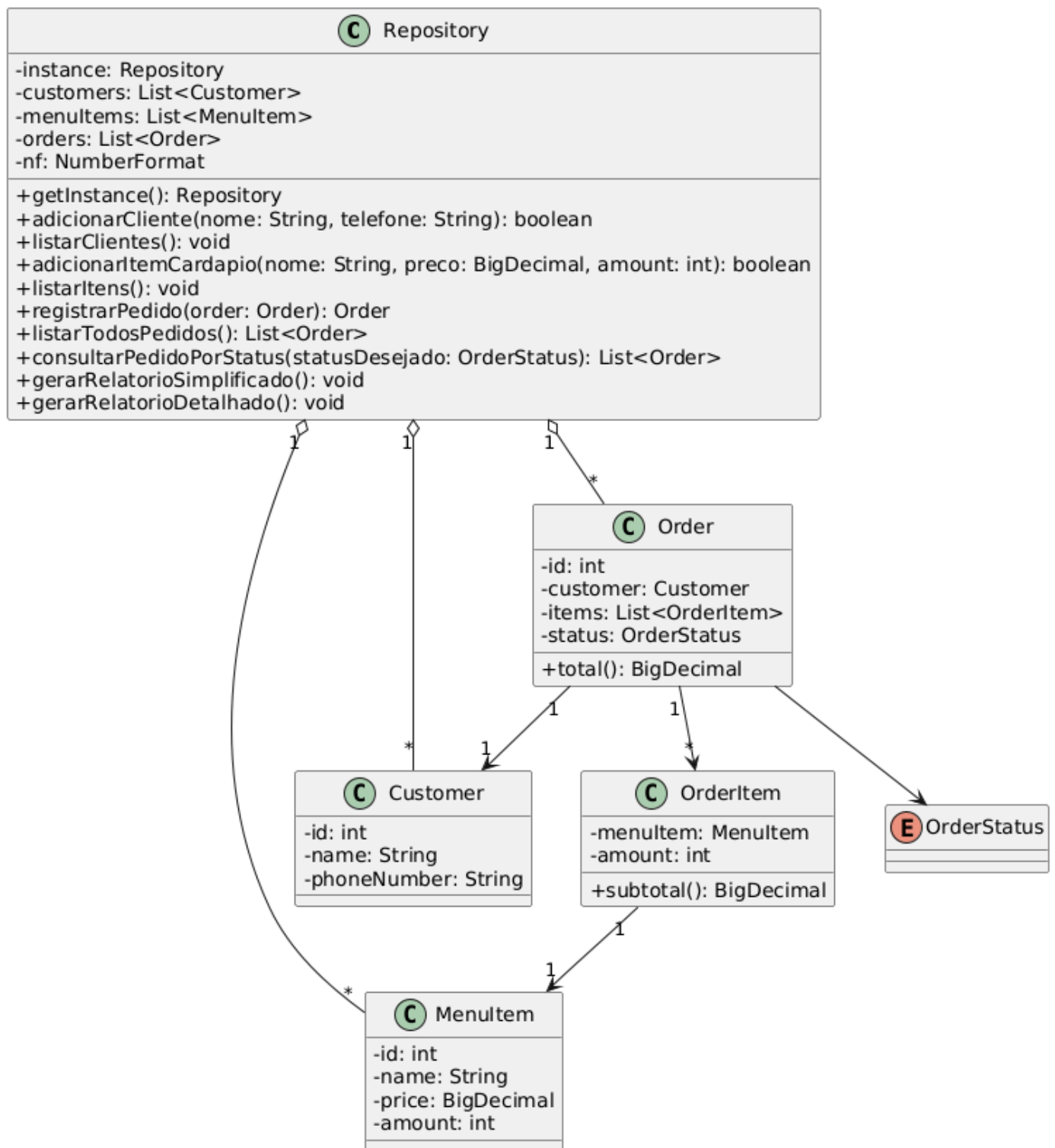
Todo o nosso trabalho foi feito em Java, e para garantir que tudo funcionasse, usamos a própria interface de linha de comando (CLI) para testar. O processo de transformar a classe Repository em Singleton foi bem direto:

```
// Mantemos a classe 'final' para evitar que alguém tente estender e quebrar nosso Singleton.
public final class Repository { 3 usages  & Luan Vinicius +1

    // Aqui guardamos a nossa única instância do Repository.
    private static volatile Repository instance; 3 usages

    // --- Nossas listas internas que armazenam os dados do sistema ---
    private final List<Customer> customers; 6 usages
    private final List<MenuItem> menuItems; 6 usages
    private final List<Order> orders; 11 usages
    private final NumberFormat nf = NumberFormat.getCurrencyInstance(new Locale( language: "pt", country: "BR"))

    // Construtor privado: aqui garantimos que ninguém fora da classe consiga criar novas instâncias.
    private Repository() { 1 usage  & Luan Vinicius
        this.customers = new ArrayList<>();
        this.menuItems = new ArrayList<>();
        this.orders = new ArrayList<>();
        System.out.println("Repository inicializado. Pronto para uso!");
    }
}
```



Entendendo o problema, percebemos que, se cada parte do sistema tivesse sua própria lista de clientes e pedidos, seria uma bagunça garantida. Precisávamos de um lugar único e confiável para armazenar tudo.

Refatoramos a classe para seguir o padrão Singleton. Trancamos o construtor (private), criamos o método getInstance() como a única forma de acessar o repositório e, para segurança, marcamos a classe como final para que ninguém pudesse estendê-la.

```
// --- Método de acesso global ---  
// Aqui fizemos a lógica do Singleton: qualquer parte do sistema que precisar do Repository vai usar  
public static synchronized Repository getInstance() { no usages  @Luan Vinicius  
    if (instance == null) {  
        instance = new Repository();  
    }  
    return instance;  
}
```

Testes e validação: Depois da mudança, rodamos a TiaLuCLI e testamos todas as funcionalidades. Adicionamos clientes e itens do cardápio, registramos pedidos com diferentes status, testamos a listagem de clientes, itens e pedidos, e geramos relatórios no console para conferir se tudo estava correto.

```
=== TiaLu Delivery ===  
1 - Adicionar cliente  
2 - Listar clientes  
3 - Adicionar item ao cardápio  
4 - Listar itens do cardápio  
5 - Registrar pedido  
6 - Consultar pedidos por status  
7 - Listar todos os pedidos  
8 - Gerar relatório simples  
9 - Gerar relatório detalhado  
0 - Sair  
Escolha uma opção: 3  
Nome do item: COCA COLA 3L  
Preço do item: 15  
Quantidade em estoque: 2  
Item 'COCA COLA 3L' adicionado ao cardápio!
```

```
=== TiaLu Delivery ===  
1 - Adicionar cliente  
2 - Listar clientes  
3 - Adicionar item ao cardápio  
4 - Listar itens do cardápio  
5 - Registrar pedido  
6 - Consultar pedidos por status  
7 - Listar todos os pedidos  
8 - Gerar relatório simples  
9 - Gerar relatório detalhado  
0 - Sair  
Escolha uma opção: 1  
Nome do cliente: João Soares  
Telefone do cliente: 75940028922  
Cliente João Soares adicionado com sucesso!
```

```
// Mantemos a classe 'final' para evitar que alguém tente estender e quebrar nosso Singleton.
public final class Repository { 3 usages  @ Luan Vinicius +1

    // Aqui guardamos a nossa única instância do Repository.
    private static volatile Repository instance; 3 usages

    // --- Nossas listas internas que armazenam os dados do sistema ---
    private final List<Customer> customers; 6 usages
    private final List<MenuItem> menuItems; 6 usages
    private final List<Order> orders; 11 usages
    private final NumberFormat nf = NumberFormat.getCurrencyInstance(new Locale( language: "pt", country: "BR"))

    // Construtor privado: aqui garantimos que ninguém fora da classe consiga criar novas instâncias.
    private Repository() { 1 usage  @ Luan Vinicius
        this.customers = new ArrayList<>();
        this.menuItems = new ArrayList<>();
        this.orders = new ArrayList<>();
        System.out.println("Repository inicializado. Pronto para uso!");
    }
}
```

4. Resultados e Discussões

O resultado de usar o Singleton na classe **Repository** foi um sistema bem mais organizado e confiável. A grande vitória foi ter centralizado tudo, garantindo que toda a aplicação acesse uma única fonte.

Antes da refatoração, cada parte do sistema podia criar seu próprio `Repository()`, gerando duplicação de dados e dificultando testes. Após aplicar o Singleton, todas as partes acessam a mesma instância, garantindo dados centralizados, consistentes e manutenção mais simples.

Claro, nem tudo é perfeito. Nossa principal desvantagem é que os dados ainda vivem só na memória, então, se fecharmos o programa, perdemos tudo. Além disso, as validações de dados (como checar se um nome não está em branco) ainda são bem simples e poderiam ser melhores.

4.1 Vantagens e Desvantagens da Solução

Uma das grandes vantagens do nosso sistema é que todos os dados ficam centralizados em uma única instância do `Repository`, o que ajuda a evitar inconsistências. Além disso, a classe possui métodos bem claros para adicionar, listar e consultar informações, e os relatórios, tanto os simples quanto os detalhados, ficaram prontos para uso imediato.

Por outro lado, ainda existem algumas limitações. A lógica de validação de dados é bem básica, por exemplo, não impede que nomes em branco sejam cadastrados, o que poderia ser melhorado em versões futuras.

5. Considerações Finais

Transformar o Repository em Singleton foi fundamental para o nosso projeto. Aprendemos na prática como um padrão de projeto pode resolver problemas reais, deixando o código mais organizado e fácil de manter. Conseguimos centralizar os dados, evitando inconsistências, e o próximo passo será persistir essas informações em um banco de dados para não perdê-las ao fechar o sistema.

O maior desafio foi garantir que a instância do Repository fosse criada de forma segura, mesmo quando várias ações aconteciam ao mesmo tempo. Para isso, usamos `synchronized` no método `getInstance()`, o que nos ensinou a construir um software confiável e robusto.

6. Referências

- **BLOCH, J.** *Effective Java*. 3ª edição, Addison-Wesley, 2018.
- **MARTIN, R. C.** *Código Limpo: Habilidades Práticas do Agile Software*. Alta Books, 2009.
- **REFACTORING GURU.** *Singleton Pattern*. Disponível em: <https://refactoring.guru/design-patterns/singleton>. Acesso em: 06 out. 2025.
- **GUERRA, E.** *Design Patterns com Java: Projeto orientado a objetos guiado por padrões*. Casa do Código, 2014.
- **ALURA.** *O que são Design Patterns?*. Blog da Alura. Disponível em: <https://www.alura.com.br/artigos/o-que-sao-design-patterns>.